SYSTEM DESIGN · FLIGHTS PLATFORM

Seat Pricing

Charge, refund and display by seat status

One wallet truth, one seat state, one derived total.
Seat money is charged with the booking, refunded when the supplier rejects the seat, and shown to the customer as the net amount charged.

SCOPE	VERSION	STATUS	OWNER
corefares · business · admin	1.0 · design baseline	For engineering review	Flights platform / seat ancillary

Applies to: cf.models.entities · cf.admin · cf.business · xtream.corefares

Design decision Charge upfront · refund on UnConfirmed · display = net charged

SECTION 01

Executive summary

One decision, three moving parts, one invariant

Seat money is currently calculated independently in three services, and none of them agrees with the others. corefares builds the customer total without seats; business debits the wallet without seats; corefares then displays every seat price regardless of whether it was charged or ever will be. The result is a customer-visible amount that was never debited, failed seats shown as paid, and seats that are never charged at all.

This design removes the ambiguity by giving each layer exactly one job. The **wallet ledger** owns money: what was debited and what was refunded. The **seats table** owns supplier outcome and refund state (`Status`, `IsRefunded`). **TotalFare** stops being an input and becomes a derived value: the net amount actually charged. Every breakdown field is then an explanation of that number rather than a second opinion about it.

DECISION

Charge upfront, refund on UnConfirmed, display the net charged amount. Seats are debited in the same Book call as the flight, because LCC suppliers (TBO, Atlas, TravelFusion) settle the seat inside that call. A seat that the supplier does not confirm is refunded as an individual wallet credit and immediately excluded from the displayed total. A failed flight keeps the existing full-order refund; per-seat refunds are suppressed in that case to avoid paying the customer twice.

1 Charge	2 Refund	3 Display	4 Guardrail
Seat prices from the server-side cached seat map are added to TotalFare and debited with the flight. The client total is validated, never trusted.	UnConfirmed seats are refunded once per seat, keyed by a deterministic transaction number, then flagged IsRefunded.	TotalFare = fare + confirmed seats + pending seats. UnConfirmed is excluded even if the refund has not landed yet.	The wallet is the source of truth. OrderAmount stays gross and immutable; net is derived, not rewritten.

What this buys

- The customer is never shown a seat as charged unless it was debited, and never shown a refused seat as chargeable.
- Agency balance is checked against the full amount, including seats, before the supplier is paid.
- Seat money is reconciled from ledger entries and seat flags, so support can answer “what did we actually charge?” without reading three code paths.
- Refunds are idempotent and safe to retry from any polling path, callback, or background job.

Scope boundary: this design changes seat money only. Fare rules, ancillary pricing other than seats, supplier contracts, and the full-order refund path are unchanged except where they intersect the per-seat refund guard.

SECTION 02

Current state and root causes

Why the three layers disagree today

Each layer computes a total from a different input set at a different moment. None of the three is wrong in isolation; the defect is that there is no single owner of seat money, so the numbers cannot agree by construction.

Layer	Current behaviour	Defect	Customer impact
corefares `AirController.Book` **:2509–2514**	Builds `TotalFare = fare + (BookingFee+GST)×pax + ancillaries`. Branded path (`BrandId != null`) adds nothing.	Seats are absent from the charged total in both paths.	Seat selection is free at the moment of booking.
business `FlightController.Book` **:4820, 4913–4961**	Sets `OrderAmount = TotalFare` and debits the wallet with that amount.	Charges exactly what corefares sent, which excludes seats.	No seat revenue; margin leak on every seat sale.
business `PopulateSeatBook FromCachedSeatMapAsync` **:4415–4418**	Defaults seat `Status = Confirmed` before the supplier replies. Runs after `SaveOrderDetails` (FC:4886).	Optimistic status; persisted price is whatever the frontend sent, not the cached price.	Price tampering is possible; unsupported seats appear confirmed in the DB.
corefares `GetBooking` **:3456–3468**	Adds every seat price to the displayed total regardless of `Status`.	Display is detached from ledger reality; ignores IsRefunded entirely.	Customer sees amounts never debited, and refused seats as charged.
admin `UpdateSeatStatus FromSeatBook` **:2326**	Merges supplier status into `seats.Status`.	No refund state exists; status can move backwards from UnConfirmed to Pending.	A refunded seat can silently re-open.
Refunds	Only a full-order refund when the flight fails (FC:6391–6433).	No per-seat refund path at all.	A refused seat is money the customer never gets back.

ROOT CAUSE

The total is recomputed at three different times from three different inputs: **before** seats exist (corefares Book), **after** the client supplies them (business Book), and **after** the supplier replies (corefares GetBooking). There is no persisted definition of "what this order charged for seats", so each layer invents one. Adding a refund path without fixing ownership would add a fourth opinion, not remove the first three.

Two secondary defects compound the problem. First, `calculatedFlightFare.FirstOrDefault()` is not guaranteed to return the client-calculated row that the breakdown renders at :3548, so the summary line and the breakdown can disagree even when both are otherwise correct. Second, the seat populate step runs after the order row is written, so the DB permanently stores a client-supplied price even when a trusted cached price exists.

SECTION 03

Design principles and invariants

The rules the implementation is checked against

The following invariants are the specification. Any implementation choice that violates one of them is a bug, even if it passes the scenario tests. They are written so they can be asserted in tests and in a nightly reconciliation job.

ID	Invariant	Enforced by	Failure symptom if broken
I1	**TotalFare shown = net money charged** = debits − seat refunds (plus the existing full-order refund when applicable).	Status-aware GetBooking; refund flags.	Customer sees a balance that does not match their wallet.
I2	A seat is charged **at most once** and refunded **at most once**.	Upfront charge at Book; `IsRefunded`; deterministic refund transaction number.	Double refund or unrefundable charge.
I3	**OrderAmount is immutable gross.** Net is derived from the ledger and seat flags, never written back.	No rewrite on refund; UpdateOrderDetails preserves the gross value.	Invoicing and audit disagree with the ledger.
I4	**DisplayPrice is server-authoritative.** The client price is advisory and is overwritten from the cached seat map before persistence.	Populate before SaveOrderDetails; price validation in business.	Price tampering; charge/refund mismatch.
I5	**UnConfirmed is terminal for money.** It cannot return to Pending, and its refund cannot be reversed automatically.	Merge guard in UpdateSeatStatusFromSeatBook.	Refunded seat silently becomes chargeable again.
I6	**Only Purchase seats carry money.** Free / Included seat charge types appear in the itinerary but never in totals.	`SeatChargeType == Purchase` filter in charge and display.	Charging for bundled seats; phantom refunds.

Sequencing rules that follow from the invariants

Rule	Why
Populate seat book from the cached map before SaveOrderDetails	The DB must store the cached DisplayPrice, not the client's number. This is the persistence point for I4.
Validate the client total before the balance check and debit	A tampered or stale total must fail without moving money. Validation is cheaper than a compensating refund.
Debit before calling the supplier	The supplier captures the seat in the same call. If the agency balance cannot cover the seat, the booking must not start.
Refund after the seat status merge, never inside it	The refund decision needs the persisted supplier outcome. Merging first also makes the helper safe to call repeatedly.
Never refund per-seat when the full-order refund will run	The full refund already returns the seat money; a per-seat credit would pay it twice.

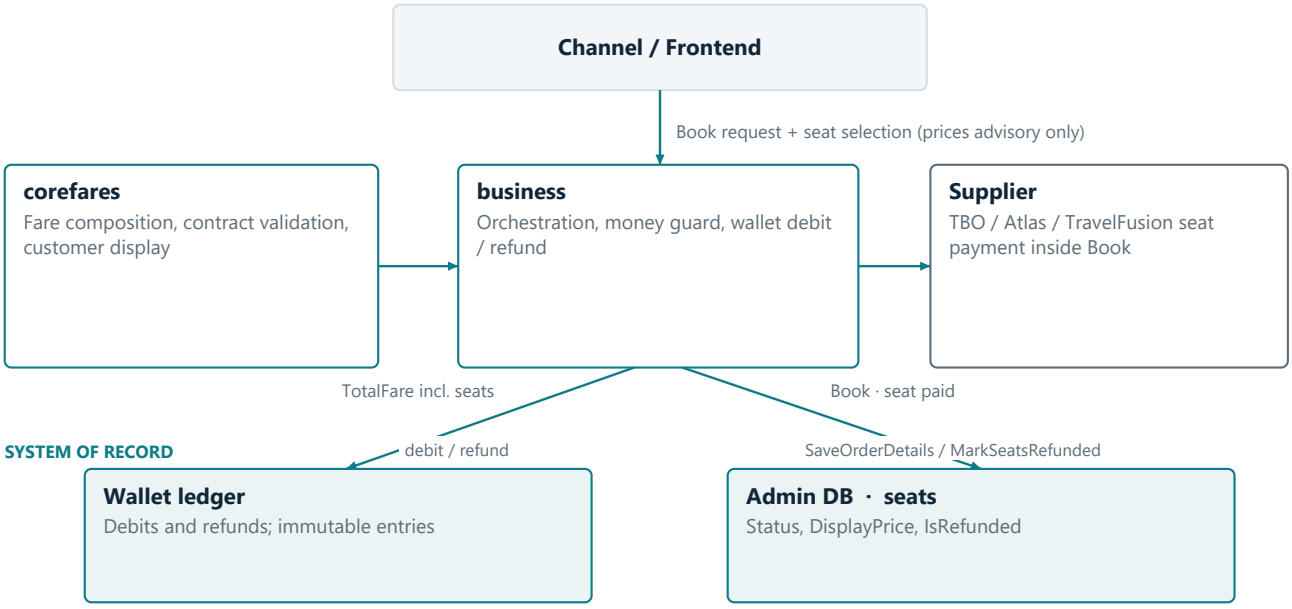
SECTION 04

Target architecture

Who owns what, and which direction money and state flow

Ownership is the design. The request path is synchronous and strict; the settlement path is asynchronous and idempotent. Nothing in the settlement path can change the amount that was originally charged.

REQUEST PATH



Money truth = wallet ledger · Seat truth = seats.Status + IsRefunded · Display = derived, never authoritative

Ownership contract

Component	Owns	Must never
corefares	Fare composition; seat amount calculation; customer-facing breakdown and TotalFare.	Trust a client seat price; debit or refund money; invent a refund.
business	Orchestration; authoritative seat price from the cached map; total validation; wallet debit and seat refund; failure mail.	Persist a client price; rewrite OrderAmount; refund a seat twice.
admin	Seat rows, Status, DisplayPrice, IsRefunded / RefundedOn / RefundTransactionNo; status merge rules.	Change DisplayPrice or IsRefunded during a status merge; re-open UnConfirmed.
Wallet ledger	Money truth: every debit and refund, keyed by TransactionNo.	Accept a duplicate transaction number as a second movement.
Supplier	Seat outcome: confirmed or unconfirmed, per segment and passenger.	Be assumed to have confirmed before it replies.

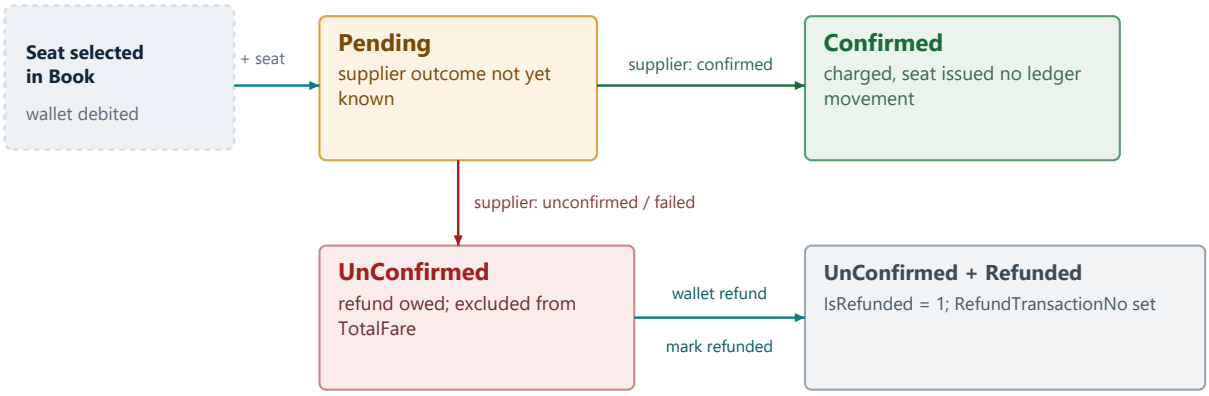
The **only** cross-boundary trust is corefares → business for the fare components. business re-derives the seat component itself and uses that number for both validation and refund. That asymmetry is deliberate: fares are commercial data owned by corefares, while seat prices are operational data owned by the cached seat map.

SECTION 05

Seat money lifecycle

Two dimensions, deliberately persisted in one row

Supplier outcome and money movement are different facts that happen at different times. The schema keeps them together in `seats` because every consumer needs one row per seat, but the implementation must treat them as orthogonal. Conflating them is how a refunded seat gets re-opened.



Guards: UnConfirmed never returns to Pending · refund is one-shot per seat · flight failure uses the full-order refund and suppresses per-seat refund

Transitions and guards

From	To	Trigger	Money effect	Guard
—	**Pending**	Seat accepted in Book	Debit Σ Purchase seat prices with the flight	Cached price exists; total validates.
Pending	**Confirmed**	Supplier confirms (merge or poll)	None; already charged	Never demote Confirmed to Pending.
Pending	**UnConfirmed**	Supplier refuses / seat failed	Refund owed; excluded from display immediately	Only for seats with Price > 0 and not yet refunded.
UnConfirmed	**UnConfirmed + Refunded**	Refund helper succeeds	One wallet credit for the batch	Deterministic transaction number; MarkSeatsRefunded is idempotent.
UnConfirmed	**Pending**	**Forbidden**	None	Merge guard rejects the transition; log and alert.
Any	**Flight failed**	Full-order refund	Full refund of OrderAmount	Per-seat helper skips: flight Failed / UnConfirmed.

WHY A BOOLEAN IS ENOUGH — AND WHERE IT STOPS

`IsRefunded` records that a refund was **posted to the ledger**, not that the supplier outcome was correct. It is intentionally one-way: there is no *refund in flight* or *refund failed* state because a failed refund leaves the flag at 0 and is retried with the same transaction number. The gap is a supplier confirming after a refund. The guard keeps the seat UnConfirmed and raises a failure mail so ops can decide whether to re-charge manually; the system never silently reverses a refund.

Pending is a charged state and stays charged. If a supplier never replies, the seat remains Pending and the customer keeps seeing it as charged, which is truthful: the money was taken. The system does not auto-refund stale pending seats, because a late confirmation would then leave the agency holding a seat it did not pay for. A reconciliation report surfaces pending seats older than the supplier SLA for ops to chase.

SECTION 06

Charge, refund and display by scenario

The contract in four rows

Scenario	Wallet	TotalFare shown	Seat line	Breakdown fields
1 · Flight booked, seat Pending	flight + seat debited	**flight + seat**	Pending — "refunded if not confirmed"	`PendingSeatAmount = seat`
2 · Flight booked, seat UnConfirmed	flight + seat debited, then seat refunded	**flight only**	Not confirmed — "refunded to wallet"	`RefundedSeatAmount = seat`
3 · Flight + seat Confirmed	flight + seat debited	**flight + seat**	Confirmed seat	`SeatAmount = seat`
4 · Flight failed	full refund (existing logic)	**0 / failed**	No per-seat line; no per-seat refund	Full refund; seat fields excluded

Two properties matter more than the individual rows

- **Row 2 is true twice.** The moment the supplier says UnConfirmed, the seat is out of TotalFare — before the refund lands. The display is a statement about what the customer owes, not about whether ops has finished its work. If the refund itself fails, the customer still is not shown the seat as charged; the failure is an operational incident handled by mail and reconciliation.
- **Row 4 suppresses row 2.** The full-order refund returns the seat money already. The per-seat helper must detect that the flight failed (or that no wallet debit exists for the order) and do nothing. This is the single most important guard against a double payout.

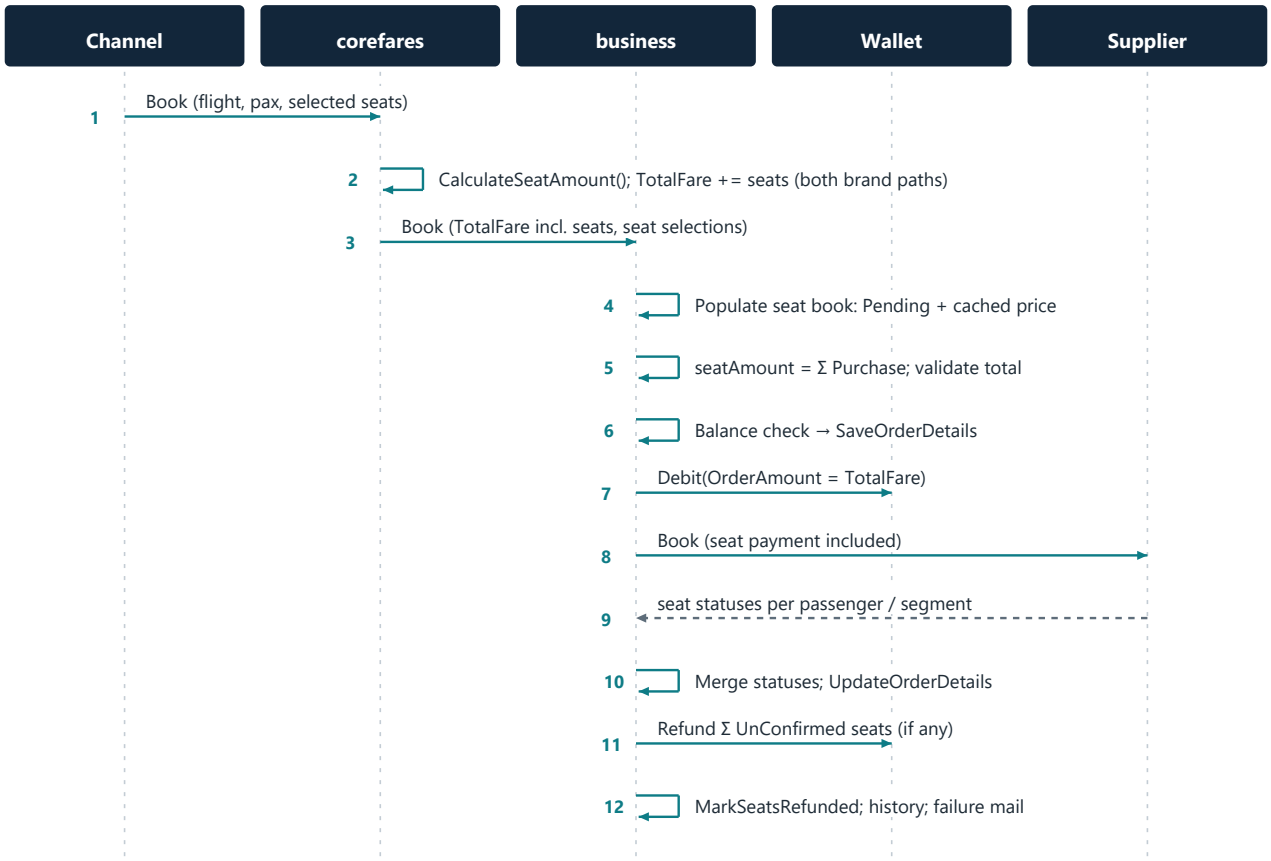
DISPLAY WORDING

Seat lines are status-aware and should name the money consequence: **Pending** — "Charged now; refunded automatically if the supplier does not confirm." **UnConfirmed** — "Not confirmed; refunded to your wallet." **Confirmed** — "Seat confirmed." Avoid the word "free" for a refunded seat; it implies the seat exists.

SECTION 07

Booking path

Charge with the flight, validate before money moves



Steps 1–7 are the synchronous contract; steps 9–12 are settlement and may also run later from polling. The diagram shows them together because the supplier often answers inside the same call.

Ordering changes in business Book

#	Change	Current anchor	Effect
1	Move `PopulateSeatBookFromCachedSeatMapAsync` before `SaveOrderDetails`.	call at FC:5045; save at FC:4886	The stored DisplayPrice is the cached server price (I4).
2	Default seat status to `Pending`, not Confirmed.	FC:4415–4418	No seat is represented as confirmed before the supplier replies.
3	Recompute `seatAmount = Σ Price where SeatChargeType == Purchase`; require the client total to match within tolerance.	before balance check (FC:4838–4875)	Server-side guard against a tampered or stale total.
4	Keep `OrderAmount = TotalFare` and the existing debit.	FC:4820, 4913–4961	Balance check and debit now cover seats with no separate transaction.
5	Call `RefundUnconfirmedSeatsAsync(orderId, correlationId)` after the status merge.	after UpdateOrderDetails (FC:6239)	Every merge path settles refunds exactly once.

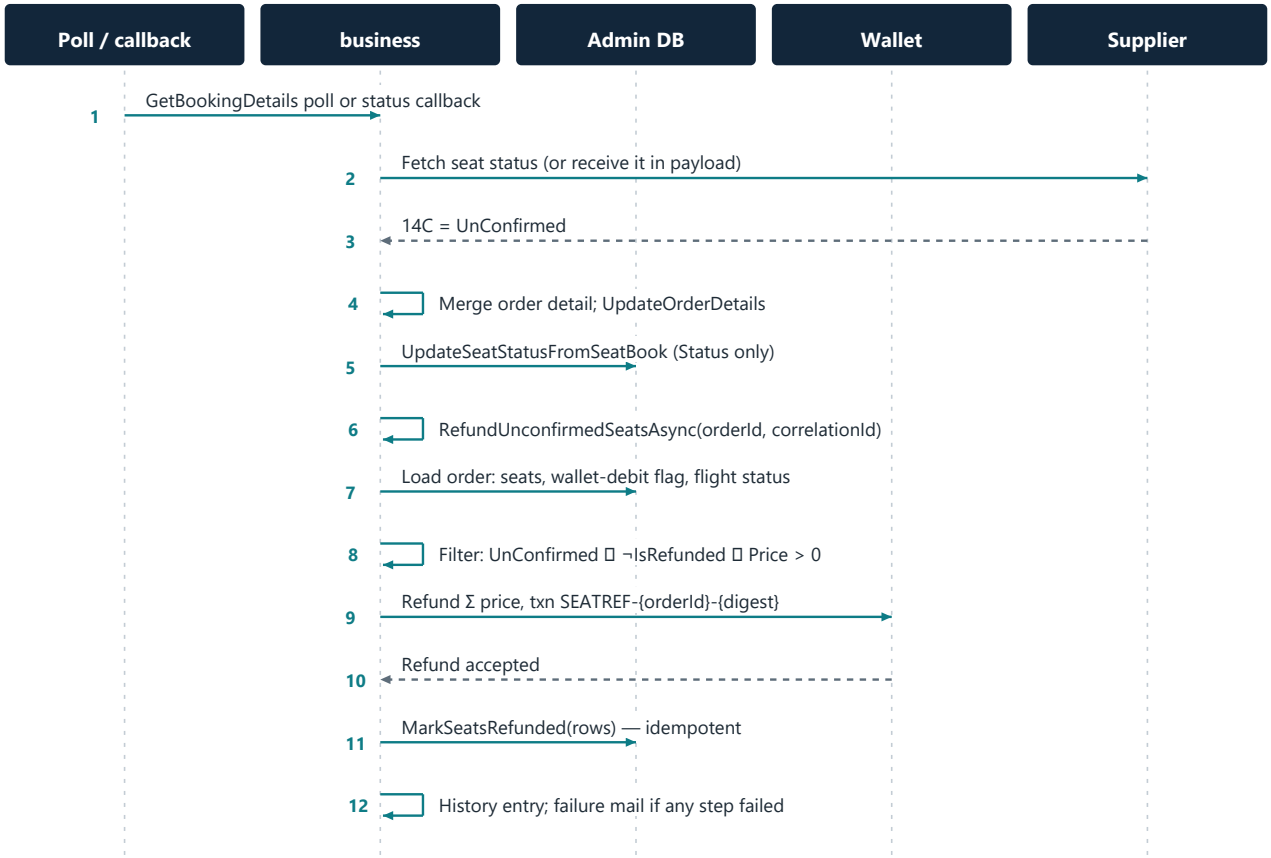
VALIDATION CONTRACT

business does not accept the client total as truth. It recomputes `seatAmount` from the cached map and compares `clientSeatComponent` against it. Because the client total also contains fare and fees, the practical check is: `abs(TotalFare − (fare + fees + seatAmount)) ≤ tolerance`, where the fare and fee components come from the same contract message. If corefares cannot send itemized components, the fallback is to require `TotalFare ≥ seatAmount` and to reject a total that is lower — but itemized is strongly preferred, because it also catches an understated fare. On mismatch, reject with `SEAT_PRICE_MISMATCH` before the balance check, and log both totals plus the seat keys. Tolerance should be one minor currency unit to absorb floating-point and rounding differences, not to absorb real pricing drift.

SECTION 08

Settlement and refund path

Idempotent, post-merge, and suppressed on flight failure



Helper contract

#	Behaviour	Failure handling
a	Load the order from `api/Order/{id}`.	Return without side effects; log.
b	**Skip** when no wallet debit exists for the order, or the flight is Failed / UnConfirmed (the full-order refund owns it).	No mail; this is an expected skip.
c	Select `Status == UnConfirmed && !IsRefunded && Price > 0`.	Empty set → no-op.

#	Behaviour	Failure handling
d	One `CreateWalletEntryAsync` (FC:16661), `WalletStatus.Refunded`, Amount = Σ price, deterministic TransactionNo.	No flags set; retry on the next poll with the same key.
e	On ledger success: `MarkSeatsRefunded` + `OrderHistoryEntryAsync` ("Seat refund: 12A, 14C – ₹X").	If marking fails, retry; the ledger key prevents a second credit. If history fails, log only.
f	Any unrecoverable error: `_paytaApiClient.SendFailureMail`.	Do not swallow; the next poll retries.

IDEMPOTENCY IS THE LINCHPIN

Order of operations is **refund first, then flag**. If the process dies between the two, the retry recomputes the same set and submits the same transaction number. That is only safe if the wallet ledger rejects a duplicate `TransactionNo`. Confirm this before implementation; if the ledger does not enforce uniqueness, add a filtered unique index on the ledger or a per-order advisory lock.

TransactionNo must be deterministic per seat set, not per attempt. A batch counter stored on the order works, but is fragile across concurrent pollers. Prefer `SEATREF-{orderId}-{sha1(sorted seat keys)[:8]}`: the same seats always produce the same key, while a later, genuinely different batch gets a new one. Do not use a random GUID or a timestamp.

Concurrent pollers can both read `IsRefunded = 0`. Either serialize per order (row lock / advisory lock) or accept both attempts and let ledger dedupe collapse them. The latter is simpler and sufficient; the former is preferable if the ledger has no uniqueness guarantee.

SECTION 09

Display and money math

TotalFare becomes a derivation, not an input

```
TotalFare = paxFare + baggage + confirmedSeat + pendingSeat

confirmedSeat = Σ seats[Status == Confirmed].Price
pendingSeat   = Σ seats[Status == Pending].Price
refundedSeat  = Σ seats[Status == UnConfirmed].Price      // excluded, shown only

paxFare = base fare + booking fee + GST, per passenger
```

The same three aggregates populate `AirClientFlightDetails`, so the total and the breakdown cannot drift. `GetBooking` is the only place that computes them; `Book` already returns `GetBooking` (:2558) and therefore inherits the behaviour without a second implementation.

Worked example — one order, three seat outcomes

Item	Amount	Status	In TotalFare	Notes
Base fare + fees (2 pax)	₹ 9,600	—	yes	From the client-calculated fare row.
Seat 12A	₹ 500	Confirmed	yes	`SeatAmount`

Item	Amount	Status	In TotalFare	Notes
Seat 15F	₹ 250	Pending	yes	`PendingSeatAmount`
Seat 14C	₹ 350	UnConfirmed, refunded	**no**	`RefundedSeatAmount`
OrderAmount (wallet debit, gross)	**₹ 10,700**	—	—	Immutable; never rewritten on refund.
Seat refund	**₹ 350**	—	—	Deterministic SEATREF key
Wallet net / total shown	**₹ 10,350**	—	—	9,600 + 500 + 250. Matches the ledger.

Two fixes in the same calculation

- `calculatedFlightFare.FirstOrDefault()` must select `CalculatedFlightFareType == "Client"`, matching the row used by the breakdown at :3548. FirstOrDefault is ordering-dependent and can silently pick a supplier or markup row.
- `PassengerSeatInfo` gains `IsRefunded`, mapped in every seat projection (:3514–3520, and the mappings at :2026, :2650, :3908, :4355, :4697). Without it, the UI cannot distinguish "refunded" from "waived" on an UnConfirmed seat.

WHY EXCLUDE UNCONFIRMED BEFORE THE REFUND LANDS

The wallet is the source of truth *for money*, but the supplier status is the source of truth *for whether the customer should pay*. Once the supplier refuses, the charge is known to be invalid; the refund is merely the bookkeeping that follows. Showing the seat as charged while the refund is in flight would overstate the customer's bill for as long as ops takes. Excluding it early is both more accurate and more honest. The corresponding ledger movement is tracked as an incident, not passed on to the customer.

SECTION 10

Data model and API contracts

New columns, DTO fields, and the refund endpoint

Schema changes are additive and nullable or defaulted, so they deploy independently of the code that reads them.

cf.models.entities · seats migration

```
ALTER TABLE seats ADD IsRefunded       bit          NOT NULL
    CONSTRAINT DF_seats_IsRefunded DEFAULT 0;
ALTER TABLE seats ADD RefundedOn       datetime2    NULL;
ALTER TABLE seats ADD RefundTransactionNo nvarchar(64) NULL;

CREATE UNIQUE INDEX UX_seats_RefundTransactionNo
    ON seats(RefundTransactionNo)
    WHERE RefundTransactionNo IS NOT NULL;
```

Surface	Additions	Purpose
Seat entity	`bool IsRefunded`, `DateTime? RefundedOn`, `string RefundTransactionNo`.	One-way refund state and audit link to the ledger entry.

Surface	Additions	Purpose
SeatBook DTO	`[NotMapped] bool IsRefunded`.	Carries refund state to the order projections without a schema change there.
PassengerSeatInfo	`bool IsRefunded`; map in every seat projection.	Customer display distinguishes refunded from merely unconfirmed.
AirClientFlightDetails	`decimal SeatAmount, PendingSeatAmount, RefundedSeatAmount`.	Breakdown explains the derived total.

New endpoint · admin

```
POST /api/Common/MarkSeatsRefunded

{
  "orderId": 88231,
  "seats": [
    { "passengerId": 17, "segmentId": 2, "seatCode": "14C",
      "transactionNo": "SEATREF-88231-a91c7f20" }
  ]
}
```

Rule	Detail
Idempotent	Skip rows already refunded. Return counts: `{ marked, skipped, notFound }`; a retry must be a no-op, not an error.
Single write	One `SaveChanges` for the batch; no per-row transactions.
Narrow surface	Sets only `IsRefunded`, `RefundedOn`, `RefundTransactionNo`. Never Status, DisplayPrice, or Price.
Keyed	Match on order + passenger + segment + seatCode. Reject a duplicate `transactionNo` on a different seat.
Authorized	Internal service identity only; never exposed to the channel.

Status merge guard · admin

Must	Must not
Update `Status` and supplier identifiers only.	Change `IsRefunded` or `DisplayPrice` during a merge.
Treat `UnConfirmed → Pending` as a rejected transition, logged.	Re-open a refunded seat because an unknown or stale status arrived.
Persist an unknown supplier status without promoting it to Confirmed.	Downgrade `Confirmed` to `Pending`.

SECTION 11

Component change list

Files, anchors, and the risk each change carries

Service	Change	Anchor	Risk / note
cf.models.entities	Add refund columns + `IsRefunded` on SeatBook (NotMapped); bump NuGet; migration on seats.	Seat entity	Low. Additive; deploy before consumers.
cf.admin	Map `IsRefunded` in `PopulateSeatBookForPassengers`; add `MarkSeatsRefunded`; add merge guard.	OrderController :431–470; CommonController :2326	Medium. Merge guard changes existing status behaviour.
cf.business	Reorder population before save; default Pending; validate total; add refund helper; call after merges and in polling.	FlightController :4415, :4820, :4886, :5045, :6239, :10040	High. Money path; ship behind a flag with strict validation last.
xtream.corefares	Add `CalculateSeatAmount` to Book (both brand paths); status-aware GetBooking; add breakdown fields and IsRefunded; fix Client fare row.	AirController :2426–2449, :2509–2514, :3456–3468, :3548	High. Display and charge must change together.

Call sites for the refund helper

The helper is safe to call repeatedly, so the rule is: call it after *every* point that can change a seat outcome. Missing a call leaves money with the agency until the next poll; calling it twice costs one no-op. Deliberately over-call rather than under-call.

Path	Where	When
Book settlement	after `UpdateOrderDetails`, FC:6239 (merges at FC:6125–6235)	TB / AT / TF seat statuses returned inside Book.
Polling	`GetBookingDetails`, FC:10040	Every poll where seat status could have changed.
Background / status job	any caller of `UpdateFlightDetailAndOrderdetailStatus`	Callback reconciliation, supplier retries, operational fixes.

SECTION 12

Failure modes and edge cases

What can go wrong, and which invariant contains it

#	Failure	System behaviour	Invariant
1	Supplier confirms after the seat was refunded.	Merge guard keeps UnConfirmed; no re-charge; failure mail for manual resolution.	I2, I5
2	Ledger refund posted, MarkSeatsRefunded fails.	Retry recomputes the same seat set and same TransactionNo; ledger dedupes; flags then set.	I2
3	MarkSeatsRefunded succeeds, history write fails.	Log only; refund already recorded. No money impact.	—
4	Two pollers refund the same seats concurrently.	Deterministic key collapses to one ledger entry; alternatively a per-order lock serializes.	I2
5	Client tampers with DisplayPrice or TotalFare.	Cached price overwrites; total mismatch rejects before debit.	I4
6	Supplier Book fails after the debit.	Existing full-order refund runs; per-seat helper skips because the flight is Failed.	I1, I2
7	Seat map cache miss for a Purchase seat.	Reject the seat selection (fail closed) rather than charge an unknown price.	I4
8	Free / Included seat selected.	Shown in itinerary; excluded from charge, refund, and totals.	I6
9	Supplier never responds; seat stays Pending.	Remains charged and displayed; reconciliation report after SLA; no auto-refund.	I1
10	Partial refusal: 2 of 3 seats UnConfirmed.	One refund entry for the refused subset only; confirmed and pending remain charged.	I1, I2
11	Second, later refusal after an earlier refund.	New batch with a new digest-based key; earlier refund untouched.	I2
12	Order has no wallet debit (pay-later / agency credit).	Helper skips; seat money follows that order's settlement path, not this one.	I3

LEGACY ORDERS ARE THE REAL EDGE CASE

Seats sold before this change were **confirmed but never charged**. After deployment, a status-aware `GetBooking` would show those seats as charged — a sudden increase with no debit behind it, exactly the bug this design removes. Do not retro-charge. Introduce a per-order `SeatPricingVersion` (1 = legacy, 2 = charged with the booking) and include seat amounts in totals only for version 2. If a schema change on orders is not acceptable, use an effective-date cutoff in configuration. In either case, mark legacy seats as out of scope in the reconciliation job so they do not generate a permanent mismatch.

SECTION 13

Rollout and rollback

Order of deployment matters more than the code

The charge and the display must switch together. If corefares includes seats while business does not charge, the customer is billed less than shown; if business charges while corefares excludes seats, the customer is billed more than shown. Both are visible failures, so use one shared flag and a single release window.

Phase	Deploy	Contains	Exit criterion
0	**cf.models.entities**	Migration: three nullable/defaulted columns, unique refund index. No behaviour.	Migration applied to all environments; backfill default 0.
1	**cf.admin**	IsRefunded mapping, MarkSeatsRefunded, merge guard. Backward compatible.	Endpoint contract tested; merge guard deployed dark.
2	**cf.business**	Pending default, cached-price ordering, refund helper, validation in *warn* mode behind `SeatPricingV2`.	No mismatch rejections; helper no-ops with zero eligible seats.
3	**xtream.corefares**	Seat amount in TotalFare (both paths), status-aware display, breakdown fields, Client fare row. Flag on.	Charge and display agree on synthetic orders; ledger net equals shown total.
4	**business**	Switch validation from warn to strict.	24h with zero SEAT_PRICE_MISMATCH; refund failure rate zero.
5	**admin / ops**	Reconciliation dashboard and refund-failure alert.	Daily reconciliation green for 7 days; flag cleanup scheduled.

Rollback

Turn `SeatPricingV2` off in both corefares and business. The flag must revert the *pair*: seat amount removed from TotalFare and per-seat refunds paused. Do not leave refunds running against a total that no longer includes seats. Charges already made and refunds already posted are not reversed by the rollback; they remain valid ledger entries and the reconciliation report continues to account for them. Keep the admin endpoint and columns in place during rollback — they are additive and harmless.

Data and migration safety

- Migration is additive only: no column rewrite, no data movement, no downtime. The filtered unique index builds on an empty set of non-null values.
- Historical orders are version 1 and excluded from seat totals; no retro-charge, no retro-refund.
- The refund transaction number is capped at 64 characters and is deterministic, so it can be recomputed after a crash without persisting attempt state.

SECTION 14

Observability and reconciliation

Prove the invariant in production, not just in tests

Every seat money event carries `orderId`, `correlationId`, the affected seat keys, the amount, and the transaction number. That tuple is enough to answer any support question and to rebuild a refund decision.

Signals

Signal	Type	Alert threshold	Meaning
`seat_price_mismatch_total`	counter	> 0 after strict rollout	Tampering, stale client, or version skew.
`seat_refund_total`	counter / amount	anomaly vs bookings	Refund volume by supplier and route.
`seat_refund_failed_total`	counter	> 0	Ledger or admin failure; customer excluded from total but not yet paid.
`seat_refund_skipped_total`	counter	spike	Full-order refund suppression or no debit; expected in bursts.
`pending_seat_age_hours`	gauge	> supplier SLA	Seats charged but never resolved; ops chase.
`seat_merge_rejected_total`	counter	> 0	Attempted UnConfirmed → Pending; investigate supplier feed.

Daily reconciliation

One query per order, comparing the ledger against the derived display. Differences are incidents, not noise.

```
expected_net = OrderAmount − Σ successful seat refunds
displayed    = paxFare + baggage + Σ Confirmed.Price + Σ Pending.Price

ASSERT  expected_net == displayed           for SeatPricingVersion = 2
ASSERT  no seat has IsRefunded = 1 and Status = Confirmed
ASSERT  no seat has RefundTransactionNo but IsRefunded = 0
```

The third assertion is the cheap canary for a half-completed refund: a transaction number without a flag means the process died between the ledger write and the admin write, and the next retry will collapse it. Alert on any row that stays in that state for more than one poll cycle.

SECTION 15

Test strategy

Cover the invariant, not just the happy path

Level	Cases	What it proves
Unit	`CalculateSeatAmount` includes Purchase only; status-aware totals; refund filter; deterministic key generation; merge guard transition table.	The arithmetic and the state rules, without infrastructure.
Contract	MarkSeatsRefunded: first call marks, second is a no-op; mismatched transaction number rejected; only three fields change.	Idempotency at the boundary.
Integration · charge	Happy path with seats; branded path; mixed Purchase/Free; client total tampered; cache miss; insufficient balance including seats.	Seats are charged with the flight and bad totals never move money.
Integration · refund	UnConfirmed at Book; UnConfirmed on later poll; partial refusal; second refusal wave; flight failure with unconfirmed seats (no double refund); ledger failure then retry.	Refunds are exactly once, correctly scoped, and suppressed when the full refund owns the money.
Integration · display	All three statuses on one order; refund posted; refund failed; legacy order excluded; Client fare row selected when multiple calculated fares exist.	TotalFare equals the ledger net in every state.
Concurrency	Two simultaneous settle calls; poll overlapping Book settlement.	No duplicate refund and no lost refund.
Migration	Apply to a copy with existing seats; defaults; unique index on partial nulls.	Deploy is safe and legacy rows stay untouched.

REQUIRED ASSERTIONS IN EVERY INTEGRATION TEST

1. `wallet_net == TotalFare` for the order.
2. `refund_count(seat) ≤ 1` per seat.
3. `OrderAmount` unchanged before and after settlement.
4. `DisplayPrice` equals the cached map value, not the request value.

Add the two cross-layer assertions to the existing end-to-end suite rather than creating a parallel one: the failure this design removes was precisely a cross-layer disagreement, so a test that only inspects one layer cannot catch a regression.

SECTION 16

Alternatives considered

Why upfront charging wins for this supplier mix

Option	Charge	Refund	Verdict
Upfront, per-seat refund (chosen)	With the flight, same Book call	On UnConfirmed, one ledger credit per batch	Selected: matches LCC settlement, protects balance, display stays truthful.
Charge on confirmation	After the supplier confirms	Not needed; no charge to reverse	Rejected: supplier has already been paid. Low balance after capture becomes an agency loss, and the balance check cannot cover a seat the customer expects.
Charge at ticket issuance	With the ticket, after settlement	On cancellation only	Rejected: seat is paid to the supplier inside Book; delaying the debit creates an exposure window with no benefit.
Order-level refund only	With the flight	Refund the whole order if any seat fails	Rejected: refunds confirmed seats and the flight; forces rebooking and over-refunds.
Display gross, refund silently	With the flight	On UnConfirmed, no display change	Rejected: violates I1; the customer sees a charge that no longer exists.
Store refund state on the order	n/a	Aggregate flag	Rejected: loses per-seat granularity needed for partial refusals and support.

The decision hinges on a property of the supplier mix, not on preference: TBO, Atlas, and TravelFusion capture the seat payment in the Book call. Any model that delays the debit shifts settlement risk to the agency. Charging upfront and refunding precisely the refused seats is the only model where the wallet, the supplier payment, and the customer total all describe the same event.

SECTION 17

Definition of done and open questions

What must be true before this ships

Definition of done

- Seat amount is included in TotalFare for branded and non-branded bookings, and the wallet debit equals TotalFare.
- The persisted DisplayPrice is always the cached server price; no request-supplied price reaches the DB.
- Seats default to Pending and never appear Confirmed before the supplier replies.
- A price mismatch rejects the booking before any wallet movement, with a stable error code.
- An UnConfirmed seat is refunded exactly once, excluded from TotalFare immediately, and flagged with a transaction number.
- The merge guard rejects UnConfirmed → Pending and never changes refund fields or price.
- The full-order refund path and the per-seat refund path cannot both pay for the same seat.
- Legacy orders are excluded from seat totals and from reconciliation noise.
- Daily reconciliation is green for seven consecutive days, with zero refund failures.

Open questions to close before implementation

#	Question	Owner	Why it blocks
1	Does the wallet ledger enforce uniqueness on `TransactionNo`?	Wallet / business	If not, refund idempotency must be added there or serialized per order.
2	Can corefares send itemized fare + fees + seat components to business?	corefares	Determines whether validation is exact or a lower-bound check.
3	Is a per-order `SeatPricingVersion` acceptable?	Data / orders	The cleanest way to exclude legacy orders from totals.
4	What is the supplier SLA for a Pending seat before ops intervenes?	Ops	Sets the pending-age alert and the reconciliation cadence.
5	What happens when a seat is cancelled by the customer before confirmation?	Product	Needs a refund trigger distinct from supplier UnConfirmed.
6	Are seats ever priced in a currency other than the order currency?	Commercial	Affects tolerance and the refund amount source.
7	Should a supplier confirmation arriving after refund trigger automatic re-charge?	Ops / product	Current design: no. Manual resolution with failure mail.

SUMMARY

Charge the seat with the flight, from the cached price. Persist Pending, not Confirmed. Refund UnConfirmed exactly once, keyed deterministically, after the status merge. Derive TotalFare from confirmed plus pending seats, and keep OrderAmount as the immutable gross. The wallet is the source of truth; the display is a view of it; the seat row is the bridge. Every rule in this document exists to keep those three statements true at the same time.